

Chapter 7: Flow-control Constructs

Mohamed M. Sabry Aly
N4-02c-92

CE/CZ-1106 2020 ©Mohamed M. Sabry Aly

1

1

Learning Objectives

- Describe how SWITCH constructs can be implemented efficiently for consecutive narrow and random wide cases.
- Contrast the implementations of pre and post-test loop constructs like WHILE, DO-WHILE and FOR.

CE/CZ-1106 2020 ©Mohamed M. Sabry Aly

2

2

Switch Statement

- How is the **SWITCH** construct implemented?
- The assembly code produced varies between compilers and depends on the nature and range of the case values.
- Two different SWITCH scenarios are examined:

```
switch(x) {
  case 0 : {S0};
           break;

  case 1 : {S1};
           break;

  case 2 : {S2};
           break;

  case 3 : {S3};
}
```

Running values
narrow range

```
switch(x) {
  case 1 : {S0};
           break;

  case 10 : {S1};
            break;

  case 100 : {S2};
             break;

  case 1000 : {S3};
}
```

Random values
wide range

CE/CZ-1106 2020 ©Mohamed M. Sabry Aly

3

3

Switch – Running & Narrow Values

```
switch(x)
{
  case 0:
    {S0};
    break;

  case 1:
    {S1};
    break;

  case 2:
    {S2};
    break;

  case 3:
    {S3};
}
```

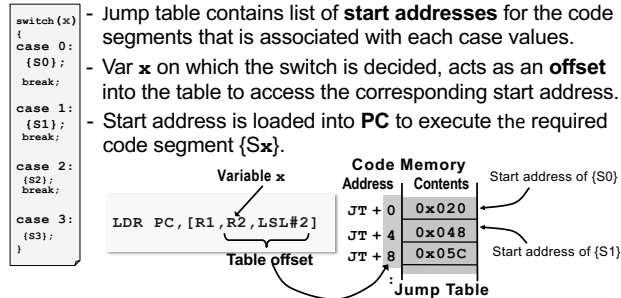
CE/CZ-1106 2020 ©Mohamed M. Sabry Aly

4

4

Switch – Running & Narrow Values

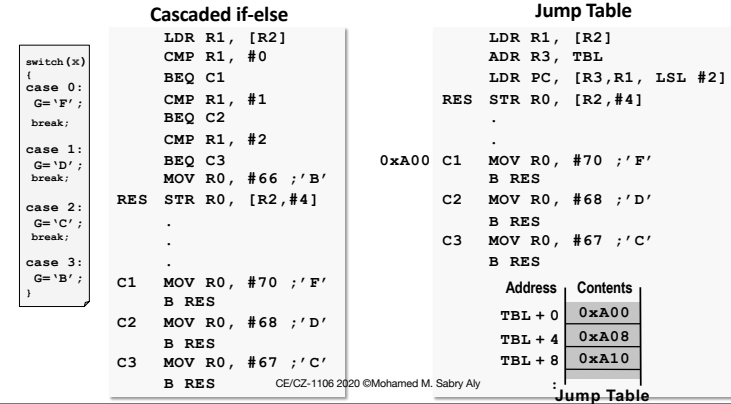
- If cases are consecutive narrow range values, a **Jump Table** is used to avoid testing each case in turn.



Note: Time taken is on average less than the equivalent if-else-if cascade and is independent of number of cases in the switch construct.

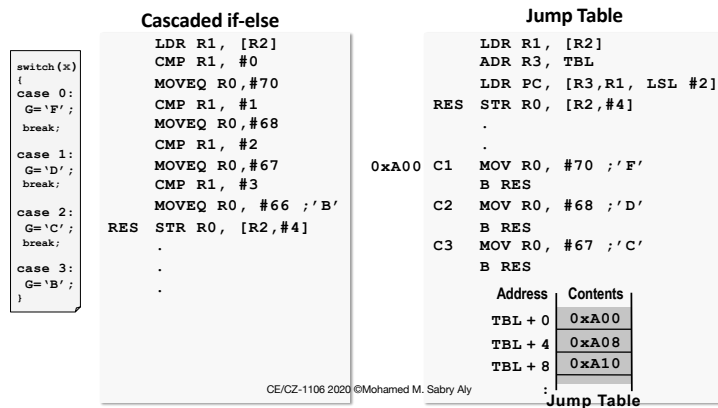
5

Jump Table Example



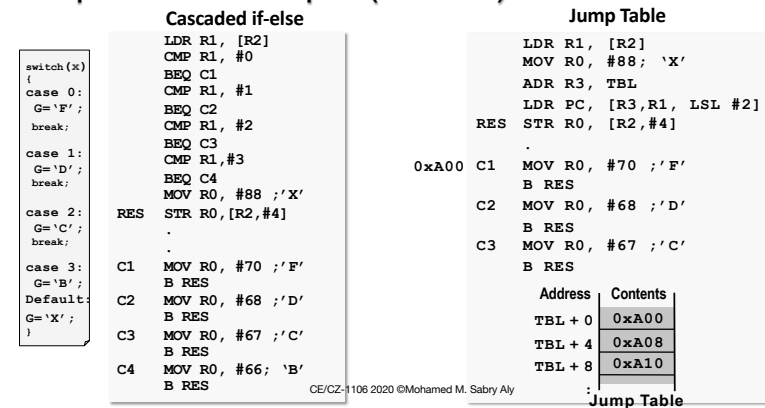
6

Jump Table Example (Conditional Execution)



7

Jump Table Example (Default)



8

Jump Table Example (Default)

Cascaded if-else	Jump Table
<pre>switch(x) { case 0: G='F'; break; case 1: G='D'; break; case 2: G='C'; break; case 3: G='B'; Default: G='X'; }</pre>	<pre>LDR R1, [R2] CMP R1, #0 BEQ C1 CMP R1, #1 BEQ C2 CMP R1, #2 BEQ C3 CMP R1, #3 BEQ C4 MOV R0, #88 ;'X' STR R0, [R2, #4] RES C1 MOV R0, #70 ;'F' B RES C2 MOV R0, #68 ;'D' B RES C3 MOV R0, #67 ;'C' B RES C4 MOV R0, #66 ;'B' B RES</pre>

0xA00	RES	LDR R1, [R2]	MOV R0, #88; 'X'
		CMP R1, #3	BGT RES
		ADR R3, TBL	LDR PC, [R3, R1, LSL #2]
		STR R0, [R2, #4]	.
C1	MOV R0, #70 ;'F'	B RES	
C2	MOV R0, #68 ;'D'	B RES	
C3	MOV R0, #67 ;'C'	B RES	
C4	MOV R0, #66 ;'B'	B RES	

Address	Contents
TBL + 0	0xA00
TBL + 4	0xA08
TBL + 8	0xA10

Jump Table

CE/CZ-1106 2020 ©Mohamed M. Sabry Aly

9

Example

- Create a subroutine that estimates the grading statistics of grades
 - Input: array of grades: 'A','B','C','D', passes in characters
 - The array terminates with a grade of 0
- Calculate statistics
 - Number of top grading marks
 - Number of B-grading marks
 - Other passing grades
- Pass address of grade array in stack, return three parameters also in stack

CE/CZ-1106 2020 ©Mohamed M. Sabry Aly

10

10

Example

- Create a subroutine grades
 - Input: array of grade
 - The array terminates
- Calculate statistics
 - Number of top grading
 - Number of B-grading
 - Other passing grade

```
Cal_grades (char* arr, int* top, int* Bgrading, int *other)
{
    *top=0;
    *Bgrading=0;
    *other=0;
    int done=0;
    int index=0;
    do{
        char val=arr[index++];
        switch (val):
        {
            case ('A'):
                *top++;
                break;
            case ('B'):
                *Bgrading++;
                break;
            case ('C'):
                *other++;
                break;
            case ('D'):
                *top++;
                break;
            default:
                done=1;
        }
    }while (done==0);
}
```

CE/CZ-1106 2020 ©Mohamed M. Sabry Aly

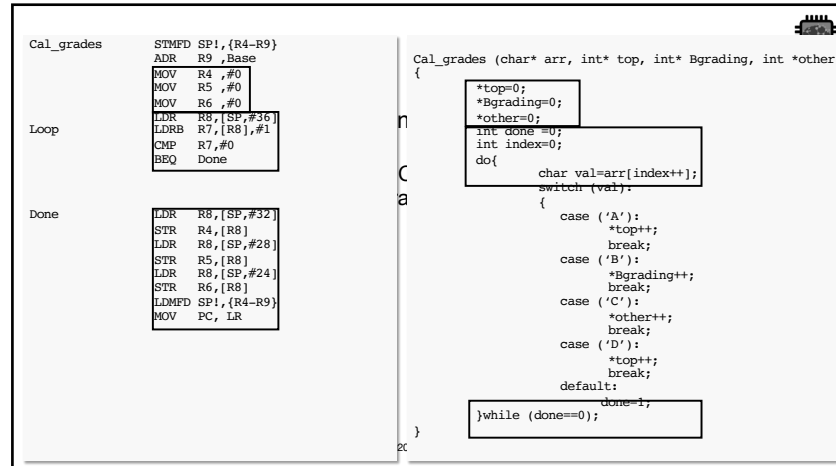
11

```
Cal_grades
    STMFD SP!, {R4-R9}
    ADR R9, Base
    MOV R4, #0
    MOV R5, #0
    MOV R6, #0

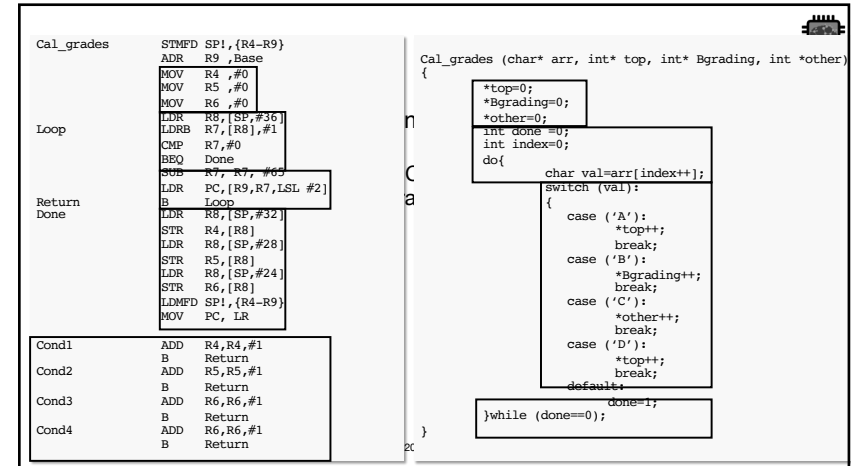
    Cal_grades (char* arr, int* top, int* Bgrading, int *other)
    {
        *top=0;
        *Bgrading=0;
        *other=0;
        int done=0;
        int index=0;
        do{
            char val=arr[index++];
            switch (val):
            {
                case ('A'):
                    *top++;
                    break;
                case ('B'):
                    *Bgrading++;
                    break;
                case ('C'):
                    *other++;
                    break;
                case ('D'):
                    *top++;
                    break;
                default:
                    done=1;
            }
        }while (done==0);
    }

    LDMFD SP!, {R4-R9}
    MOV PC, LR
```

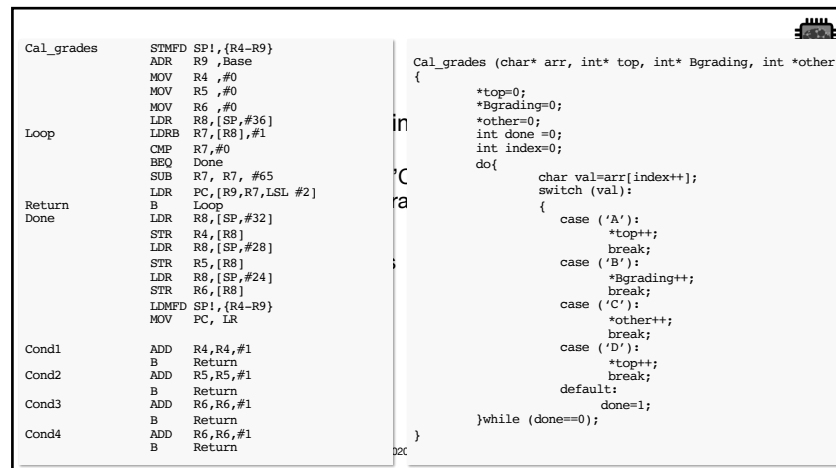
12



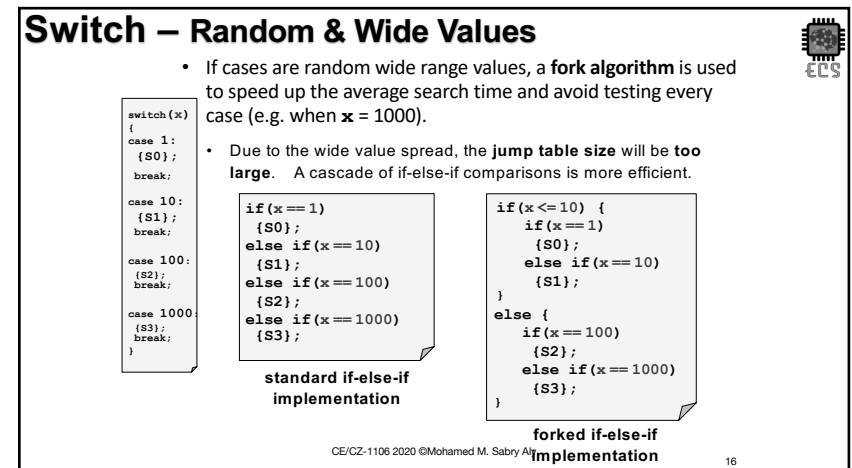
13



14



15



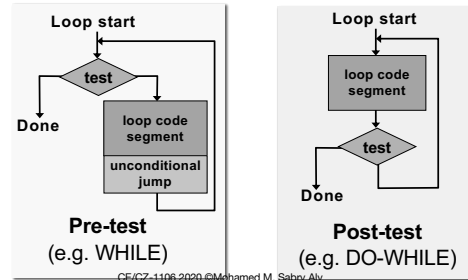
16

Loops

Loop constructs are distinguished by the position of their conditional test.

Pre-test loop may **never execute** its loop code segment.

Post-test loop executes the loop segment **at least once**.



CE/CZ-1106 2020 ©Mohamed M. Sabry Aly

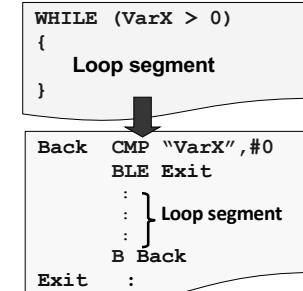
17

17

WHILE Implementation

- Implementation of the **WHILE** loop constructs:

- This is an example of a **pre-test** loop.
- If the condition (**VarX > 0**) is false, the loop segment is not executed at all.



Note: **VarX** is variable. you will need to load it to a register.

Implementation of WHILE in ARM assembly language

CE/CZ-1106 2020 ©Mohamed M. Sabry Aly

18

18

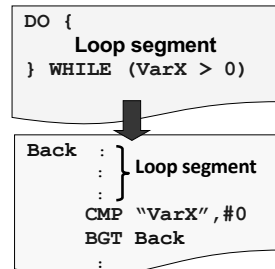
DO-WHILE Implementation

- Implementation of the **DO-WHILE** loop constructs
- This is an example of a **post-test** loop.

- The loop segment is executed at least once before condition is tested.

- Post-test loop construct is **more efficient** than the pre-test as there is no need for an additional unconditional jump.

Note: **VarX** is variable. you will need to load it to a register.



Implementation of DO- WHILE in ARM assembly language

CE/CZ-1106 2020 ©Mohamed M. Sabry Aly

19

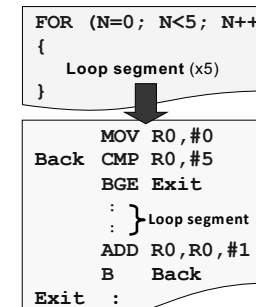
19

FOR Implementation

- Implementation of **FOR** loop constructs:

- The FOR loop is a **pre-test** loop that evaluates the condition first before executing loop segment.

- If loop segment is executed and count **N** is not used in loop segment, some optimizing compilers implement the **FOR** loop using a **post-test with decrement & test for zero**.



Implementation of FOR in ARM assembly language

CE/CZ-1106 2020 ©Mohamed M. Sabry Aly

20

20

Example: Summation 1 to N



```
Sum = 0;
for (i=0; i<=N; i++)
    Sum=Sum+i;
```

Pre-test

```

Loop    MOV    R2, #0
        MOV    R0, #0
        CMP    R2, #N
        BGT    END
        ADD    R0, R0, R2
        ADD    R2, R2, #1
        B      Loop
END     STR    R0, [ "sum" ]
```

Post-test

```

Loop    MOV    R2, #N
        MOV    R0, #0
        ADD    R0, R0, R2
        SUBS   R2, R2, #1
        BNE    Loop
        STR    R0, [ "sum" ]
```

CE/CZ-1106 2020 ©Mohamed M. Sabry Aly

21

21

Example: Summation 1 to N



```
Sum = 0;
for (i=0; i<=N; i++)
    Sum=Sum+i;
```

Pre-test

```

Loop    MOV    R2, #0
        MOV    R0, #0
        CMP    R2, #N
        BGT    END
        ADD    R0, R0, R2
        ADD    R2, R2, #1
        B      Loop
END     STR    R0, [ "sum" ]
```

Post-test

```

Loop    MOV    R2, 0
        MOV    R0, #0
        ADD    R0, R0, R2
        ADD    R2, R2, #1
        CMP    R2, #5
        BLE    Loop
        STR    R0, [ "sum" ]
```

CE/CZ-1106 2020 ©Mohamed M. Sabry Aly

22

22

Summary



- **SWITCH** constructs can be implemented efficiently depending nature of the case values.
- Narrow consecutive values can benefit from a jump table.
- Forked if-else-if can be used with wide ranged values.
- **Post-test** loops are **more efficient** than pre-test loops for the same loop segment.
- With optimised compilers, the low-level code produced may not tally directly with the high-level operations. (e.g. loop increments may be implemented as decrements for better code efficient).

CE/CZ-1106 2020 ©Mohamed M. Sabry Aly

23

23